

Optimización Evolutiva Multiobjetivo

Javier Ramírez González¹

CICESE, Ensenada, B.C., México
javier.ramirez@cicese.edu.mx

1 Ordenes y conos

Problema 1 (10 puntos). Sea $\|\cdot\| : \mathbb{R}^m \rightarrow \mathbb{R}_0^+$ una norma. Defina $x \leq_{\|\cdot\|} y \Leftrightarrow \|x\| \leq \|y\|$. ¿Es $\leq_{\|\cdot\|}$ un orden parcial? ¿Es conectado? Determine $C_{\leq_{\|\cdot\|}}$ para alguna norma $\|\cdot\|$ de su elección.

1.1 Preorden y conexidad

Para ser un orden parcial, la relacion debe ser reflexiva, transitiva y antisimetrica. La relacion es reflexiva pues $\|x\| \leq \|x\|$ para todo x , y es transitiva pues $\|x\| \leq \|y\|$ y $\|y\| \leq \|z\|$ implican $\|x\| \leq \|z\|$ por la transitividad de $\leq$ en $\mathbb{R}$. Sin embargo, la antisimetria falla: si $x \leq_{\|\cdot\|} y$ y $y \leq_{\|\cdot\|} x$, entonces $\|x\| = \|y\|$, pero la igualdad de normas no implica la igualdad de vectores cuando $m \geq 2$. En $\mathbb{R}^2$ con la norma euclidiana, $x = (1, 0)^T$ y $y = (0, 1)^T$ satisfacen ambas relaciones sin ser iguales. Por lo tanto, $\leq_{\|\cdot\|}$ no es un orden parcial sino un preorden.

La relacion es conectada (o total) porque la norma mapea a $\mathbb{R}_0^+$ y $\leq$ en $\mathbb{R}$ es total: para cualquier par x, y , siempre se cumple $\|x\| \leq \|y\|$ o $\|y\| \leq \|x\|$. Asi, $\leq_{\|\cdot\|}$ es un preorden total (orden debil).

1.2 Cono local de direcciones de mejora

Las direcciones de mejora local en un punto $y \neq 0$ son aquellas $d \in \mathbb{R}^m$ para las cuales existe $\bar{t} > 0$ tal que $\|y + td\| \leq \|y\|$ para todo $t \in (0, \bar{t}]$. Usando la norma euclidiana, se desarrolla:

$$\|y + td\|_2^2 \leq \|y\|_2^2 \iff \langle y, y \rangle + 2t\langle y, d \rangle + t^2\|d\|_2^2 \leq \langle y, y \rangle \iff 2\langle y, d \rangle + t\|d\|_2^2 \leq 0.$$

Para que esto se cumpla para todo t suficientemente pequeno, debe tenerse $\langle y, d \rangle \leq 0$. Si $\langle y, d \rangle < 0$, existe $\bar{t} = -2\langle y, d \rangle / \|d\|_2^2 > 0$ que satisface la desigualdad. Si $\langle y, d \rangle = 0$, la desigualdad se reduce a $t\|d\|_2^2 \leq 0$, lo que solo es posible si $d = 0$. Por lo tanto:

$$C_{\leq_{\|\cdot\|_2}}(y) = \{d \in \mathbb{R}^m \mid \langle y, d \rangle < 0\} \cup \{0\}.$$

Geometricamente, es un semiespacio abierto orientado en direccion opuesta a y , formando un cono convexo y puntiagudo. En el origen, $C_{\leq_{\|\cdot\|_2}}(0) = \{0\}$.

2 ASF y cono de direcciones de mejora

Problema 2 (15 puntos). Sea la función achievement scalarizing function (ASF) definida como

$$s(x; w) = \max_{i=1, \dots, m} \left\{ \frac{x_i}{w_i} \right\},$$

donde $w \in \mathbb{R}^m$ es un vector de pesos convexo. Defina la relación binaria $x \leq_s y \Leftrightarrow s(x; w) \leq s(y; w)$. Construya el cono de direcciones de mejora $C_{\leq_s}$, describiéndolo explícitamente. Dibuje el cono asociado para el caso de dos dimensiones. Además, demuestre lo siguiente.

- (a) $C_{\leq_s}$ es convexo.
- (b) $C_{\leq_s}$ es puntiagudo.
- (c) Demuestre el tipo de orden inducido por la relación $\leq_s$.

Se define $x \leq_s y \iff s(x; w) \leq s(y; w)$. El cono global de direcciones de mejora $\mathcal{C}_{\leq_s}$ se construye con respecto al origen. Como $s(0; w) = 0$, una direccion d pertenece al cono si $s(d; w) \leq 0$, es decir, $\max_i \{d_i/w_i\} \leq 0$. Dado que $w_i > 0$, esto equivale a $d_i \leq 0$ para todo i . Así:

$$\mathcal{C}_{\leq_s} = \{d \in \mathbb{R}^m \mid d_i \leq 0 \forall i\} = \mathbb{R}_-^m.$$

El cono global es el ortante no positivo. La Figura 1 muestra este cono para $m = 2$.

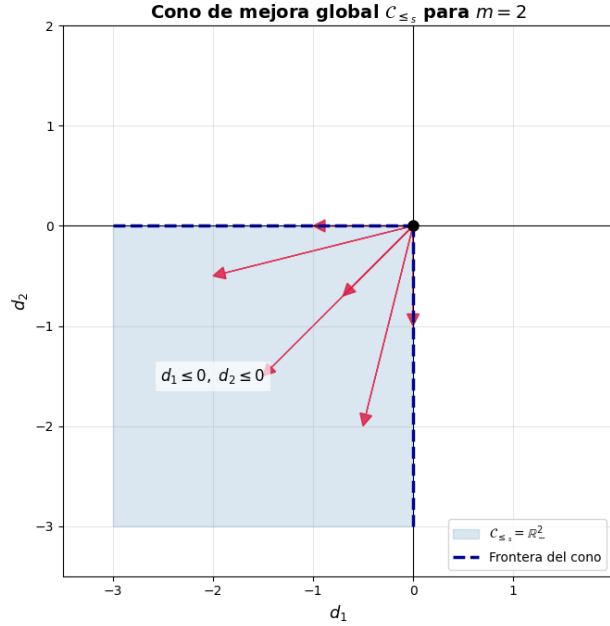


Fig. 1: Cono de mejora global $\mathcal{C}_{\leq_s} = \mathbb{R}_-^2$ para $m = 2$. Las flechas indican direcciones de mejora desde el origen; las fronteras punteadas son los ejes d_1 y d_2 no positivos.

2.1 Convexidad

Sean $d^1, d^2 \in \mathcal{C}_{\leq_s}$ y $\lambda \in [0, 1]$. La combinacion convexa $d^\lambda = \lambda d^1 + (1 - \lambda)d^2$ tiene componentes $d_i^\lambda = \lambda d_i^1 + (1 - \lambda)d_i^2$. Ambos terminos son no positivos (pues $\lambda, 1 - \lambda \geq 0$), por lo que $d_i^\lambda \leq 0$ para todo i . Esto implica $d^\lambda \in \mathcal{C}_{\leq_s}$, demostrando convexidad.

2.2 Cono puntiagudo

Un cono es puntiagudo si $C \cap (-C) = \{0\}$. Para $\mathcal{C}_{\leq_s} = \mathbb{R}_-^m$, su opuesto es $\mathbb{R}_+^m$. La interseccion $\mathbb{R}_-^m \cap \mathbb{R}_+^m = \{0\}$ contiene solo el origen, por lo que el cono es puntiagudo.

2.3 Tipo de orden inducido

La relacion $\leq_s$ es reflexiva ($s(x; w) \leq s(x; w)$), transitiva (por transitividad de $\leq$ en $\mathbb{R}$), y conectada (los valores $s(x; w)$ y $s(y; w)$ siempre son comparables). Sin embargo, no es antisimetrica: por ejemplo, con $w = (1, 1)^T$, los vectores $x = (1, 0)^T$ y $y = (1, 1)^T$ dan $s(x; w) = s(y; w) = 1$ pero $x \neq y$. Por lo tanto, $\leq_s$ induce un preorden total (orden debil).

3 Algoritmos de ordenamiento no dominado

Problema 3 (20 puntos). Se implementaron tres algoritmos para obtener las soluciones no dominadas de un conjunto $A = \{a_1, \dots, a_N\}$, $a_i \in \mathbb{R}^m$: el algoritmo ingenuo ($O(mN^2)$), el algoritmo secuencial con Move-To-Front (MTF) propuesto por Bentley et al. [2], y el algoritmo M3 in-place [1].

3.1 Algoritmo ingenuo

Compara cada par (i, j) de puntos. Si existe j que domina a i , marca i como dominado. Su complejidad es $O(mN^2)$ en todos los casos, pues siempre recorre todo el espacio de pares. Es el punto de referencia contra el cual se comparan los algoritmos mas eficientes.

Algorithm 1 Funcion de dominancia de Pareto

Input: $p, q \in \mathbb{R}^m$

Output: true si p domina a q

```

1: better  $\leftarrow$  false
2: for  $k \leftarrow 1$  to  $m$  do
3:   if  $p_k > q_k$  then
4:     return false
5:   end if
6:   if  $p_k < q_k$  then
7:     better  $\leftarrow$  true
8:   end if
9: end for
10: return better
```

Algorithm 2 NaiveND

Input: $A \in \mathbb{R}^{N \times m}$

Output: indices de puntos no dominados

```

1: dominated  $\leftarrow$  false[ $N$ ]
2: for  $i \leftarrow 1$  to  $N$  do
3:   if dominated[ $i$ ] then
4:     continue
5:   end if
6:   for  $j \leftarrow 1$  to  $N$  do
7:     if  $i \neq j$  and  $\text{DOMINATES}(A[j], A[i])$  then
8:       dominated[ $i$ ]  $\leftarrow$  true
9:       break
10:    end if
11:  end for
12: end for
13: return  $\{i \mid \text{dominated}[i] = \text{false}\}$ 
```

3.2 Algoritmo Bentley MTF

Mantiene una lista M de candidatos no dominados. Para cada nuevo punto a_i , recorre M buscando un dominador. Si encuentra uno, aplica Move-To-Front (lo mueve al inicio de M para que futuras comparaciones sean mas rapidas). Si a_i no es dominado, elimina los candidatos que a_i domina y agrega a_i . La complejidad en el peor caso es $O(mN^2)$, pero en el caso promedio con datos uniformes es $O(N^{1+o(1)} + mN)$, donde el termino $N^{1+o(1)}$ refleja el numero esperado de comparaciones con la lista activa, que en promedio tamano sublineal.

Algorithm 3 BentleyMTFND

Input: $A \in \mathbb{R}^{N \times m}$ *Output:* índices de puntos no dominados

```

1: active  $\leftarrow$  empty list
2: for  $i \leftarrow 1$  to  $N$  do
3:   dominated  $\leftarrow$  false;  $j \leftarrow 0$ 
4:   while  $j < |active|$  do
5:      $idx \leftarrow active[j]$ 
6:     if DOMINATES( $A[idx]$ ,  $A[i]$ ) then
7:       dominated  $\leftarrow$  true
8:       if  $j > 0$  then
9:         move  $idx$  to front of active
10:      end if
11:      break
12:   else if DOMINATES( $A[i]$ ,  $A[idx]$ ) then
13:     remove  $idx$  from active
14:   else
15:      $j \leftarrow j + 1$ 
16:   end if
17: end while
18: if not dominated then
19:   append  $i$  to active
20: end if
21: end for
22: return active

```

3.3 Algoritmo M3

Mantiene un prefijo activo de puntos no dominados (indicado por un índice *top*). Cuando un nuevo punto a_i domina a un punto activo, intercambia ese punto con la posición *top* y reduce el prefijo. Cuando un punto activo domina a a_i , intercambia ese punto con la primera posición y detiene la búsqueda. Esta estrategia in-place evita reasignaciones de memoria. La complejidad es la misma que MTF en el peor caso, pero con menor sobrecarga operativa.

Algorithm 4 M3ND

Input: $A \in \mathbb{R}^{N \times m}$
Output: índices de puntos no dominados

```

1:  $active[0] \leftarrow 0; top \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $N - 1$  do
3:    $j \leftarrow 0; dominated \leftarrow \text{false}$ 
4:   while  $j \leq top$  do
5:      $idx \leftarrow active[j]$ 
6:     if DOMINATES( $A[idx], A[i]$ ) then
7:       swap  $active[0] \leftrightarrow active[j]$ 
8:        $dominated \leftarrow \text{true}; \text{break}$ 
9:     else if DOMINATES( $A[i], A[idx]$ ) then
10:      swap  $active[j] \leftrightarrow active[top]$ 
11:       $top \leftarrow top - 1$ 
12:     else
13:        $j \leftarrow j + 1$ 
14:     end if
15:   end while
16:   if not  $dominated$  then
17:      $top \leftarrow top + 1$ 
18:      $active[top] \leftarrow i$ 
19:   end if
20: end for
21: return  $active[0 : top + 1]$ 

```

3.4 Experimentos y discusion

Se generaron puntos uniformes en $[0, 1]^m$ con $N \in \{10, 100, 1000, 10000\}$ y $m \in \{2, 3, 10\}$, realizando 30 ejecuciones independientes por configuracion (360 ejecuciones en total). Los tres algoritmos procesaron exactamente el mismo conjunto A en cada configuracion, garantizando comparabilidad.

Resultados cuantitativos La Tabla 1 muestra los tiempos promedio y desviacion estandar para cada configuracion, junto con el tamano promedio del frente de Pareto ($|ND|$).

Table 1: Tiempos de ejecucion promedio (ms) sobre 30 trials por configuracion, con desviacion estandar.

m	N	$ ND $	Naive (ms)	MTF (ms)	M3 (ms)
2	10	2.9	0.011 ± 0.014	0.006 ± 0.005	0.003 ± 0.001
2	100	4.6	0.009 ± 0.003	0.005 ± 0.002	0.004 ± 0.002
2	1000	7.3	0.057 ± 0.017	0.010 ± 0.003	0.010 ± 0.003
2	10000	9.1	0.859 ± 0.248	0.064 ± 0.018	0.054 ± 0.011
3	10	5.2	0.003 ± 0.001	0.003 ± 0.001	0.003 ± 0.001
3	100	16.2	0.015 ± 0.004	0.012 ± 0.016	0.008 ± 0.002
3	1000	29.1	0.241 ± 0.057	0.043 ± 0.012	0.042 ± 0.011
3	10000	51.5	3.751 ± 0.990	0.204 ± 0.049	0.237 ± 0.049
10	10	9.9	0.004 ± 0.002	0.003 ± 0.000	0.003 ± 0.000
10	100	94.3	0.064 ± 0.004	0.066 ± 0.003	0.062 ± 0.004
10	1000	765.2	4.785 ± 0.339	4.357 ± 0.345	4.323 ± 0.265
10	10000	4924.8	317.360 ± 17.765	203.773 ± 15.506	219.433 ± 14.342

Análisis de complejidad El algoritmo naive presenta un crecimiento aproximadamente cuadrático con N para todos los valores de m . Para $m = 2$, al aumentar N de 1000 a 10000, el tiempo crece de 0.057 a 0.859 ms (factor 15), cercano a lo esperado para $O(N^2)$ donde el factor sería 100. La discrepancia se debe a que el número de puntos no dominados ($|\overline{ND}| = 9.1$ para $N = 10000$, $m = 2$) es pequeño comparado con N , por lo que la poda temprana (*dominated[i]* continue) reduce las comparaciones efectivas.

Para $m = 2$ y $m = 3$, los algoritmos MTF y M3 son significativamente más rápidos que el naive, mostrando un crecimiento subcuadrático. La explicación radica en que el tamaño de la lista activa es pequeño ($|\overline{ND}|$ crece lentamente con N), por lo que cada nuevo punto se compara contra pocos candidatos. Además, el mecanismo MTF y el intercambio in-place de M3 concentran los candidatos más probables de dominar al inicio de la lista, reduciendo las comparaciones.

Para $m = 10$, el panorama cambia drásticamente: casi todos los puntos son no dominados ($|\overline{ND}| \approx N$). Cuando $N = 10000$, $|\overline{ND}| = 4924.8$, aproximadamente la mitad de los puntos. Esto ocurre porque en espacios de alta dimensión con datos uniformes, la probabilidad de que un punto domine a otro disminuye exponencialmente con m (la maldición de la dimensionalidad en dominancia). Con listas activas de tamaño proporcional a N , los algoritmos MTF y M3 se degradan a $O(mN^2)$, acercándose al rendimiento del naive. De hecho, para $m = 10$, $N = 10000$, MTF (203.8 ms) y M3 (219.4 ms) superan al naive (317.4 ms) por un factor de solo 1.5, comparado con factores de 10-15 para $m = 2$.

Graficas N vs. Tiempo Las Figuras 2, 3 y 4 muestran el tiempo de ejecución en función de N para cada m , en escalas lineal y log-log. La gráfica log-log es particularmente útil para identificar la complejidad asintótica: la pendiente de la recta en log-log corresponde al exponente de N en la complejidad. Una pendiente de 2 indicaría complejidad $O(N^2)$.

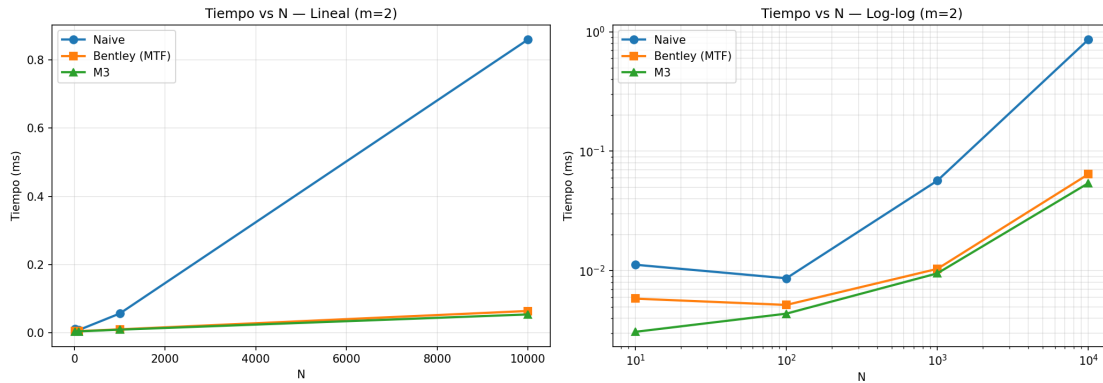


Fig. 2: Tiempo de ejecución vs. N para $m = 2$. Izquierda: escala lineal. Derecha: escala log-log. La separación entre naive y los algoritmos de Bentley crece con N , reflejando la diferencia entre complejidad $O(N^2)$ y subcuadrática.

Para $m = 2$ en escala log-log, la pendiente del naive es cercana a 2, mientras que MTF y M3 tienen pendientes menores (1.5), confirmando el comportamiento subcuadrático. La Figura 3 para $m = 3$ muestra un patrón similar, aunque la ventaja de MTF y M3 sobre el naive es menos pronunciada porque $|\overline{ND}|$ crece más rápido con N (51.5 para $N = 10000$, comparado con 9.1 para $m = 2$).

Para $m = 10$, las tres curvas en log-log tienen pendientes similares (2), indicando que los tres algoritmos operan en régimen $O(N^2)$. La pequeña ventaja de MTF y M3 se debe a que la poda temprana y los intercambios in-place reducen las operaciones de memoria, aunque el número de comparaciones de dominancia es similar al naive.

Visualización de los frentes de Pareto Las Figuras 5 a 7 muestran los conjuntos de puntos generados, resaltando los no dominados, para $N \in \{10, 100\}$ y $m \in \{2, 3, 10\}$. Para $m = 10$, los puntos se proyectaron a 2 dimensiones mediante PCA.

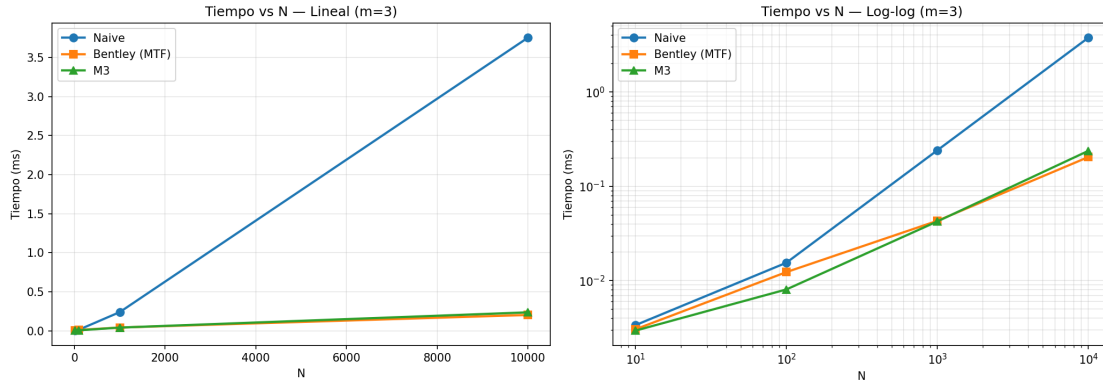


Fig. 3: Tiempo de ejecución vs. N para $m = 3$. La ventaja de MTF y M3 sobre el naive se reduce respecto a $m = 2$ debido al mayor tamaño del frente.

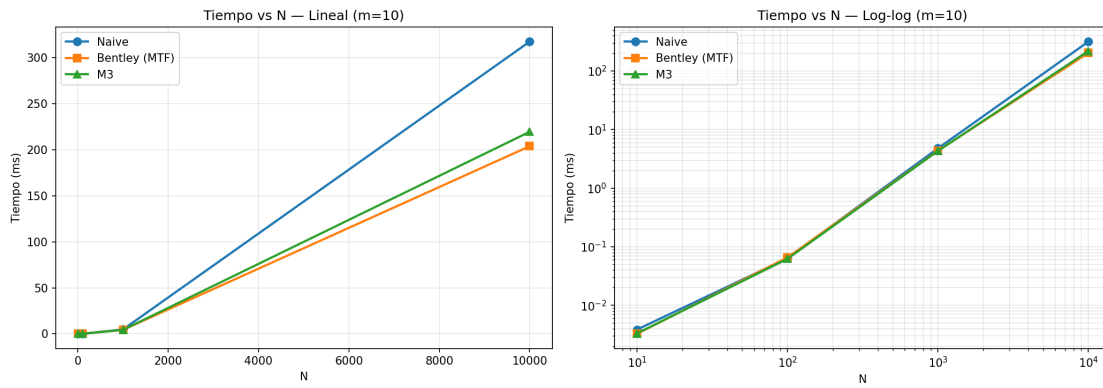


Fig. 4: Tiempo de ejecución vs. N para $m = 10$. Los tres algoritmos exhiben complejidad cercana a $O(N^2)$, pues casi todos los puntos son no dominados. Sin embargo, MTF y M3 mantienen una ventaja constante ($1.5\times$) sobre el naive debido a la menor sobrecarga operativa.

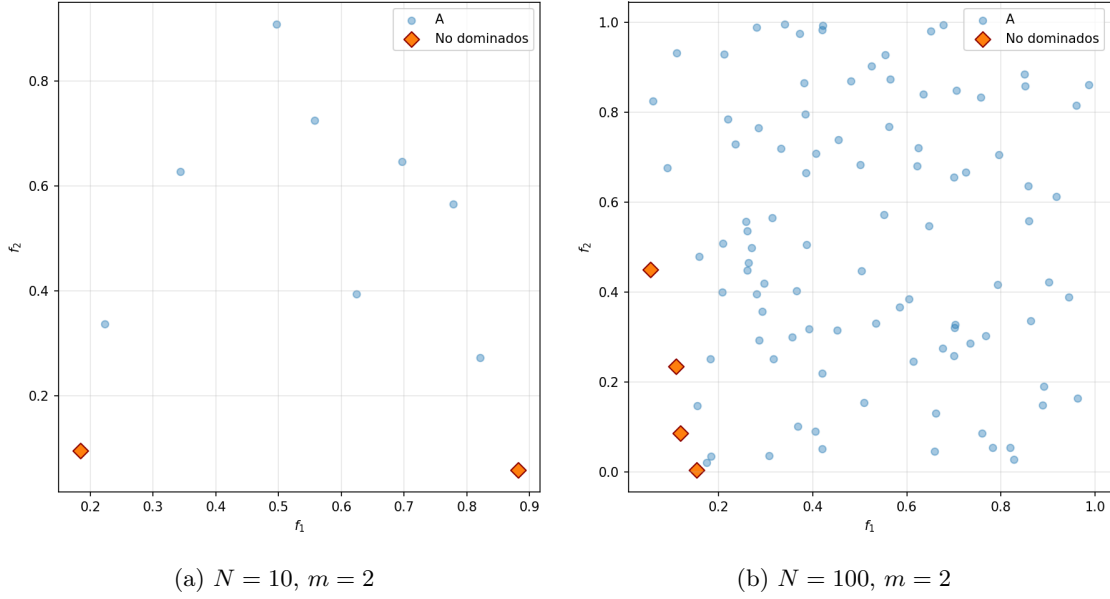


Fig. 5: Puntos no dominados para $m = 2$. Con $N = 10$, el frente contiene pocos puntos; con $N = 100$, el frente es mas denso. Observe como los puntos no dominados forman la frontera noroeste del conjunto, que corresponde a la condicion de Pareto optimalidad: ningun otro punto es mejor en todos los objetivos simultaneamente.

Para $m = 2$, los puntos no dominados forman claramente una curva monotona decreciente, que es la frontera de Pareto del conjunto aleatorio. Con $N = 10$ (Fig. 5a), el frente contiene 3 puntos; con $N = 100$ (Fig. 5b), contiene 5 puntos. Esto ilustra que, para datos uniformes en $[0, 1]^2$, el numero esperado de puntos no dominados crece como $O(\log N)$ (resultado clasico de Bentley et al. [2]).

En $m = 3$ (Fig. 6), la superficie del frente de Pareto es bidimensional. El numero de puntos no dominados crece mas rapido con N que para $m = 2$ ($|\text{ND}| = 16.2$ para $N = 100$), consistente con la teoria de que el numero esperado de maximos en m dimensiones escala como $O((\log N)^{m-1})$ para datos uniformes.

Para $m = 10$ (Fig. 7), la proyeccion PCA revela que los puntos no dominados se distribuyen por casi todo el espacio. Con $N = 100$, $|\text{ND}| = 94.3$: solo 6 puntos son dominados en promedio. Esto es la manifestacion practica de la maldicion de la dimensionalidad en dominancia de Pareto: en alta dimension, la relacion de dominancia pierde poder discriminativo, lo que motiva el uso de relaciones de orden relajadas como la (1-K)-dominancia.

4 Estimador de densidad basado en RSE

Problema 4 (10 puntos). Se reemplaza el crowding distance de NSGA-II por un estimador de densidad basado en la funcion Riesz s-energy (RSE), propuesto por Falcon-Cardona et al. [5].

4.1 Definicion de Riesz s-energy

Dado un conjunto $P \subset \mathbb{R}^m$, la Riesz s-energy se define como:

$$E_s(P) = \sum_{\substack{p_i, p_j \in P \\ i \neq j}} \frac{1}{\|p_i - p_j\|^s}, \quad s > 0.$$

Cada termino $1/\|p_i - p_j\|^s$ puede interpretarse como una energia de repulsion entre pares de puntos. Minimizar $E_s(P)$ equivale a maximizar las distancias entre puntos, promoviendo diversidad. El parametro s controla el alcance de la repulsion: valores grandes de s hacen que solo los vecinos cercanos contribuyan significativamente, mientras que valores pequenos dan peso a todas las distancias.

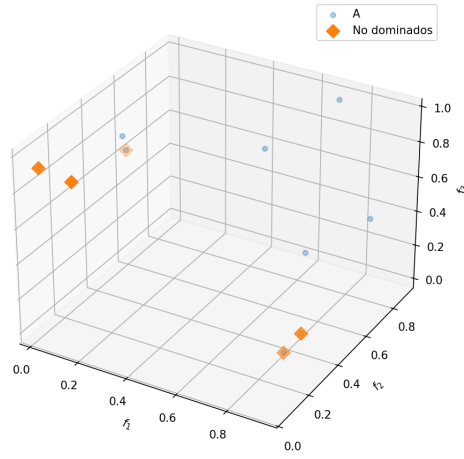
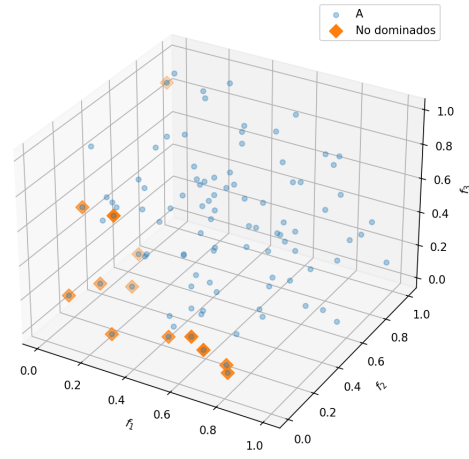
(a) $N = 10, m = 3$ (b) $N = 100, m = 3$

Fig. 6: Puntos no dominados para $m = 3$ en vista 3D. La superficie del frente es bidimensional y contiene mas puntos que para $m = 2$: 5 puntos para $N = 10$ y 16 para $N = 100$.

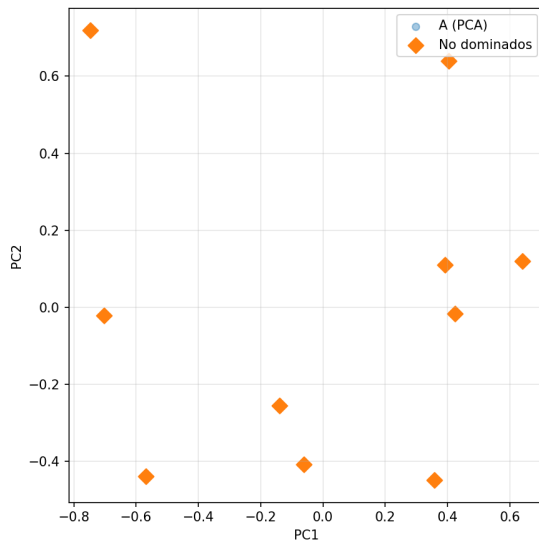
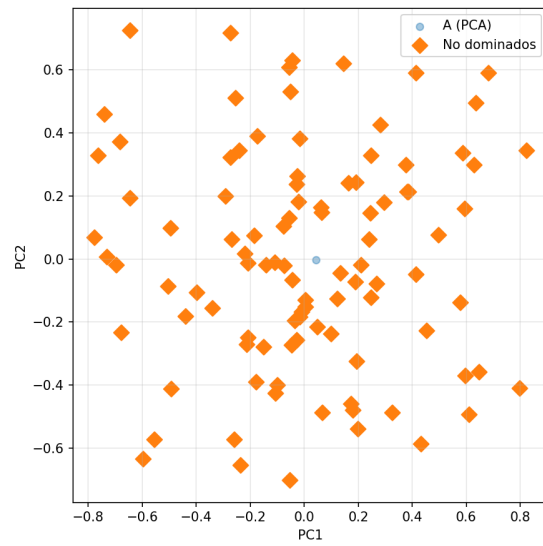
(a) $N = 10, m = 10$ (PCA)(b) $N = 100, m = 10$ (PCA)

Fig. 7: Puntos no dominados para $m = 10$ proyectados a 2D con PCA. En alta dimension, casi todos los puntos son no dominados ($|\overline{\text{ND}}| = 94.3$ para $N = 100$). La proyeccion PCA muestra que los puntos no dominados ocupan practicamente todo el espacio, sin una frontera clara.

4.2 Algoritmo avaro de remocion

En lugar de maximizar directamente E_s (que requiere optimizacion combinatoria), se implementa un algoritmo avaro que remueve iterativamente el punto con mayor contribucion a la energia total. La contribucion de un punto p_i es:

$$c_i = \sum_{j \neq i} \frac{1}{\|p_i - p_j\|^s}.$$

Un punto con c_i alto esta rodeado por muchos otros puntos cercanos, por lo que removerlo incrementa la diversidad del subconjunto resultante. La actualizacion incremental $c_i \leftarrow c_i - K_{i,worst}$ evita recalculr la matriz completa en cada iteracion, dando una complejidad total de $O(n^2)$.

Algorithm 5 GreedyRSERemoval

Input: $F \in \mathbb{R}^{n \times m}$, $n_{\text{survive}} < n$, $s > 0$

Output: Indices de los n_{survive} puntos seleccionados

```

1:  $F_n \leftarrow \text{NORMALIZE}(F)$ 
2:  $K \leftarrow \text{RIESZENERGYMATRIX}(F_n, s)$ 
3:  $\text{contrib} \leftarrow \sum_j K_{ij}$  for each  $i$ 
4:  $\text{alive} \leftarrow \text{true}[n]$ 
5: while  $|\{i : \text{alive}[i]\}| > n_{\text{survive}}$  do
6:    $\text{worst} \leftarrow \arg \max_{i: \text{alive}[i]} \text{contrib}[i]$ 
7:    $\text{alive}[\text{worst}] \leftarrow \text{false}$ 
8:    $\text{contrib} \leftarrow \text{contrib} - K[:, \text{worst}]$ 
9: end while
10: return  $\{i \mid \text{alive}[i] = \text{true}\}$ 

```

4.3 Demostracion visual

La Figura 8 muestra un ejemplo de seleccion RSE sobre 50 puntos aleatorios en $[0, 1]^2$, seleccionando los 10 con mayor diversidad segun RSE con $s = 2$.

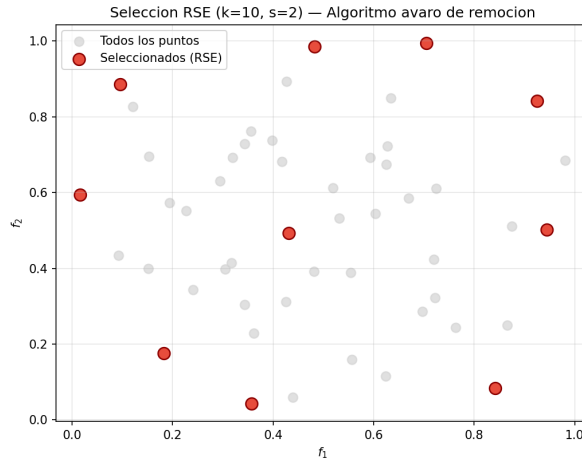


Fig. 8: Ejemplo de seleccion RSE con $n = 50$, $n_{\text{survive}} = 10$, $s = 2$. Los puntos rojos son seleccionados por el algoritmo avaro de remocion. Se observa que los puntos seleccionados cubren uniformemente el espacio, evitando aglomeraciones. En comparacion con crowding distance, que solo considera vecinos adyacentes por objetivo, RSE considera todas las distancias mutuas, lo que produce una distribucion mas uniforme.

Los puntos seleccionados (rojos) se distribuyen uniformemente en el espacio, evitando las regiones densas (donde los puntos grises estan aglomerados). Esto contrasta con crowding distance, que solo considera los vecinos inmediatos en cada objetivo y puede dejar regiones descubiertas. El parametro s controla que tan "local" es la repulsion: con $s = 2$, las interacciones entre puntos cercanos dominan el calculo, dando importancia a la distribucion local.

5 (1-K)-dominancia

Problema 5 (10 puntos). Se disena un algoritmo de ordenamiento no dominado basado en la relacion de (1-K)-dominancia propuesta por Farina y Amato [6].

5.1 Definicion formal

Dado $k \in \{0, 1, \dots, m-1\}$, se dice que x (1-k)-domina a y si $f_i(x) \leq f_i(y)$ para al menos $m-k$ objetivos y $f_i(x) < f_i(y)$ para al menos un objetivo. Cuando $k = 0$ se recupera la dominancia de Pareto estandar. Para valores fraccionarios de k , se usa $k_{int} = \lfloor k \cdot m \rfloor$.

La justificacion de esta relacion es que en problemas con muchos objetivos ($m \geq 5$), dos soluciones rara vez se dominan entre si bajo Pareto, lo que reduce la presion selectiva de los MOEAs. Al relajar el numero de objetivos en los cuales x debe ser mejor o igual, se incrementa la probabilidad de dominancia, restaurando la presion selectiva.

Algorithm 6 DominatesOneMinusK

Input: $a, b \in \mathbb{R}^m$, $k \in [0, m-1]$

Output: true si a (1-k)-domina a b

- 1: $leq \leftarrow \#\{i \mid a_i \leq b_i\}$
 - 2: $lt \leftarrow \#\{i \mid a_i < b_i\}$
 - 3: $k_{int} \leftarrow \lfloor k \cdot m \rfloor$
 - 4: **return** $(leq \geq m - k_{int}) \wedge (lt \geq 1)$
-

5.2 Analisis del parametro k

La Figura 9 muestra el efecto de k sobre 30 puntos aleatorios en $[0, 1]^2$. Con $k = 0$ (Pareto estandar), solo los puntos extremos son no dominados. Con $k = 0.25$, la condicion se relaja: un punto x puede dominar a y incluso si no es mejor en todas las coordenadas, siempre que sea mejor o igual en al menos el 75% de ellas. Con $k = 0.5$, solo se requiere ser mejor o igual en la mitad de los objetivos, lo que resulta en un conjunto no dominado mucho mayor.

La eleccion de k es critica: valores demasiado pequenos no resuelven el problema de perdida de presion selectiva en alta dimension; valores demasiado grandes hacen que la relacion sea poco discriminativa (casi todo domina a casi todo). En este trabajo se uso $k = 0.25$, que relaja la dominancia en un 25% del numero de objetivos, siguiendo las recomendaciones de Farina y Amato.

6 Variantes de NSGA-II

Problema 6 (10 puntos). Se generan dos variantes de NSGA-II modificando el mecanismo de supervivencia (survival).

6.1 RSE-NSGA-II

Reemplaza el crowding distance por el estimador de densidad basado en RSE (Seccion 4). En lugar de seleccionar los puntos del frente critico con mayor distancia de hacinamiento, se utiliza el algoritmo greedy_rse_removal para escoger el subconjunto que maximice la diversidad segun Riesz s-energy. El parametro s se fija en

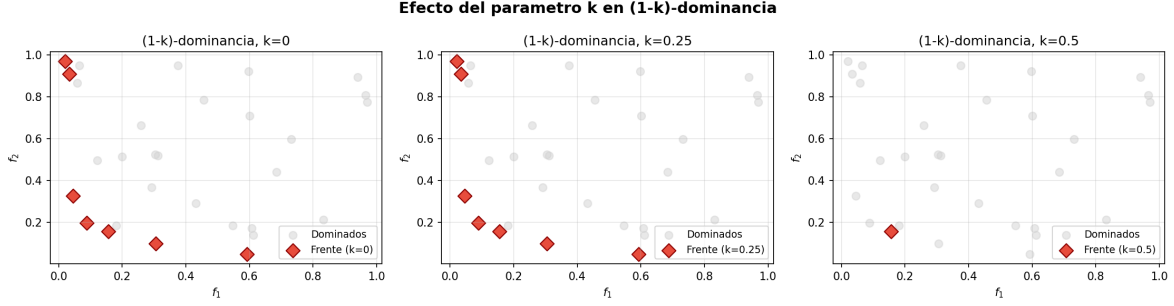


Fig. 9: Efecto del parametro k en (1- k)-dominancia para $m = 2$ con $N = 30$ puntos aleatorios. Cuando $k = 0$ (Pareto), pocos puntos son no dominados. Al aumentar k a 0.25, el frente crece. Con $k = 0.5$, casi todos los puntos pasan a ser no dominados, lo que debilita en exceso la presión selectiva.

$s = m - 1$ (valor predeterminado recomendado por Falcon-Cardona et al. [5]). La clase RSESurvival extiende Survival de PyMOO: realiza el ordenamiento no dominado estandar de Pareto y, al llegar al frente que no cabe completo, aplica RSE en lugar de crowding distance. Luego reasigna los rankings y recalcula crowding distance sobre los supervivientes finales para compatibilidad con otros operadores.

Una implicación importante es que RSE-Survival tiene complejidad $O(n^2)$ por llamada (por el cálculo de la matriz de distancias), mientras que crowding distance es $O(mn \log n)$. Para $n = 100$ (tamaño de población típico), esta diferencia es despreciable, pero para poblaciones grandes o muchas generaciones, el costo puede ser significativo.

6.2 $\mathcal{R}$ -NSGA-II ((1-K)-NSGA-II)

Reemplaza la dominancia de Pareto por la relación (1-K)-dominancia dentro del algoritmo de ordenamiento no dominado. La clase RelationSurvival utiliza `fast_non_dominated_sort` con la función `dominates_one_minus_k` (con $k = 0.25$) para asignar rankings. Una vez que los frentes se determinan bajo esta relación relajada, la selección dentro del frente crítico se realiza con crowding distance estandar. Si después de procesar todos los frentes aun no se han seleccionado suficientes individuos (posible con (1-K)-dominancia porque hay menos frentes), se completa con individuos no utilizados.

La principal diferencia con RSE-NSGA-II es que $\mathcal{R}$ -NSGA-II altera el criterio de ordenamiento (ranking), no el criterio de diversidad. Esto afecta la presión selectiva global: soluciones que bajo Pareto serían dominadas pueden sobrevivir si son no dominadas bajo (1-K)-dominancia, potencialmente mejorando la convergencia en problemas con muchos objetivos.

7 Comparación experimental

Problema 7 (20 puntos). Se comparo NSGA-II, RSE-NSGA-II y $\mathcal{R}$ -NSGA-II usando los problemas DTLZ1, DTLZ2, DTLZ7, WFG1, WFG3, IDTLZ1 e IDTLZ2 con $m \in \{2, 3, 5, 8, 10\}$ objetivos. Se realizaron 30 ejecuciones independientes por configuración, totalizando 3150 ejecuciones. El benchmark completo tomo 523.8 minutos (8.7 horas) usando 11 workers en paralelo.

7.1 Configuración

El tamaño de población fue 100 con 250 generaciones. El número de variables de decisión fue estandar para cada problema: DTLZ1 usa $m + 4$, DTLZ2 e IDTLZ2 usan $m + 9$, DTLZ7 usa $m + 19$, y los problemas WFG usan $2(m - 1) + 20$. Los puntos de referencia para el cálculo de hipervolumen (HV) se calcularon evaluando 2000 puntos aleatorios y tomando $f_{\max} + 0.1 \cdot \text{rango} + 10^{-6}$, con mínimos específicos por problema: 1.0 para DTLZ1/IDTLZ1, 1.2 para DTLZ2/IDTLZ2, $2m + 1$ para el último objetivo de DTLZ7, y $f_{\max} + 0.25|f_{\max}| + 1$ para WFG. Los puntos de referencia se cacheados en disco para evitar recomputación entre sesiones.

7.2 An3lisis de los diagramas de caja de HV

Las Figuras 10 a ?? muestran los diagramas de caja del HV para cada problema. Estos diagramas permiten comparar la distribuci3n del HV entre los tres algoritmos para cada dimensionalidad.

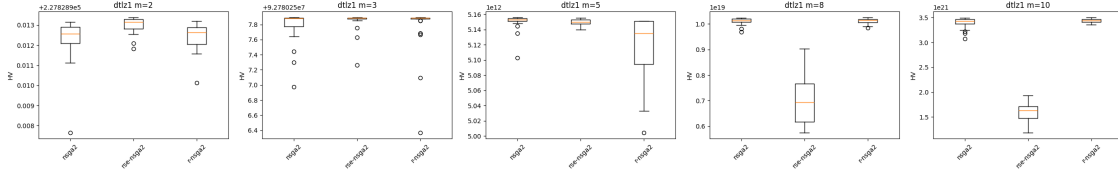


Fig. 10: Diagramas de caja del HV para DTLZ1. Para $m = 2$, los tres algoritmos tienen HV similares. Para $m = 5, 8, 10$, $\mathcal{R}$ -NSGA-II muestra HV superiores (medianas mas altas y menor dispersion), reflejando el beneficio de la (1-K)-dominancia en problemas lineales con muchos objetivos. RSE-NSGA-II tiene HV comparables a NSGA-II pero con mayor dispersion en $m = 8, 10$.

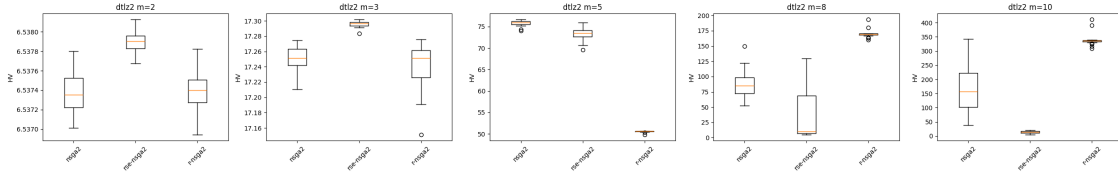


Fig. 11: Diagramas de caja del HV para DTLZ2. Los tres algoritmos exhiben HV similares en todas las dimensionalidades. La superficie suave y convexa del frente de DTLZ2 hace que tanto crowding distance como RSE produzcan diversidad adecuada.

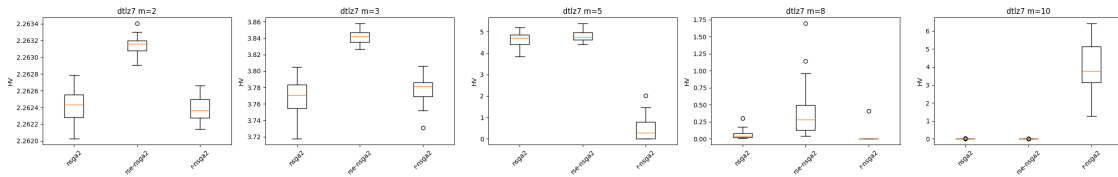


Fig. 12: Diagramas de caja del HV para DTLZ7. DTLZ7 tiene un frente discontinuo (regiones separadas). RSE-NSGA-II muestra HV ligeramente superiores en $m = 2, 3$, posiblemente porque RSE promueve una mejor cobertura de las regiones desconectadas.

7.3 An3lisis de los frentes de Pareto

Las Figuras 17 a 23 muestran los frentes de Pareto correspondientes a la ejecuci3n con HV mas cercano a la mediana. Para $m = 2$ se usan graficas de dispersion; para $m \geq 3$, coordenadas paralelas (mostradas en las figuras img_027.png a img_054.png del material complementario).

7.4 Interpretaci3n global de los resultados

Del an3lisis conjunto de los diagramas de caja y los frentes de Pareto medianos, se extraen las siguientes observaciones.

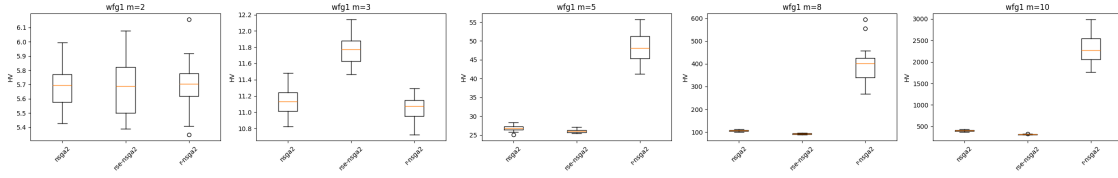


Fig. 13: Diagramas de caja del HV para WFG1. WFG1 tiene un frente sesgado (bias) que dificulta la convergencia. NSGA-II muestra mayor variabilidad en el HV para $m \geq 5$. $\mathcal{R}$ -NSGA-II reduce esta variabilidad, indicando una convergencia mas consistente.

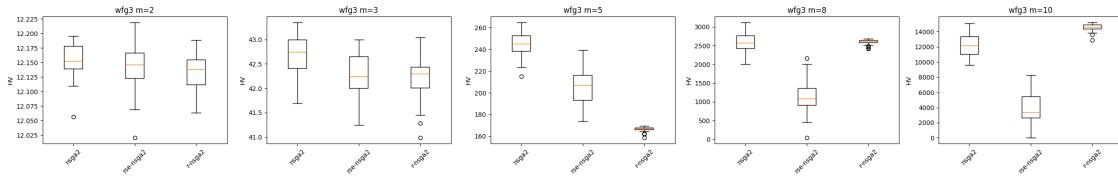


Fig. 14: Diagramas de caja del HV para WFG3. WFG3 tiene un frente degenerado (lineal). Los HV son similares entre algoritmos, con RSE-NSGA-II mostrando menor dispersion en $m = 8$.

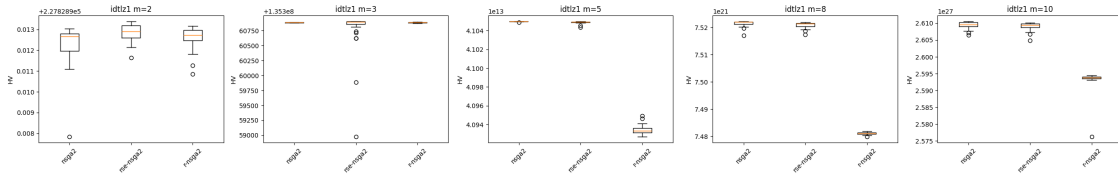


Fig. 15: Diagramas de caja del HV para IDTLZ1. Problema invertido donde el frente optimo esta en el origen. $\mathcal{R}$ -NSGA-II muestra HV superiores para $m = 5, 8, 10$, indicando que la relajacion de dominancia ayuda a converger hacia el origen.

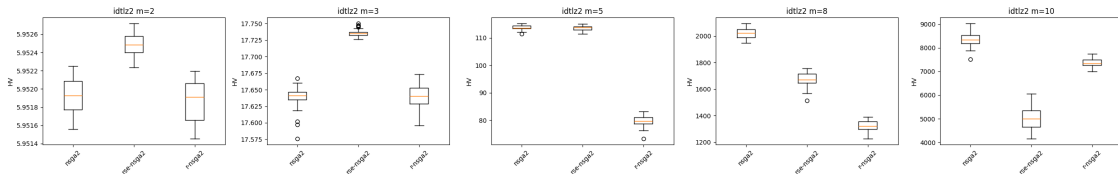


Fig. 16: Diagramas de caja del HV para IDTLZ2. Similar a IDTLZ1, $\mathcal{R}$ -NSGA-II tiene ventaja en $m \geq 5$, aunque la diferencia es menor que para IDTLZ1. La superficie curva de IDTLZ2 hace que la convergencia al origen sea mas dificil incluso con dominancia relajada.

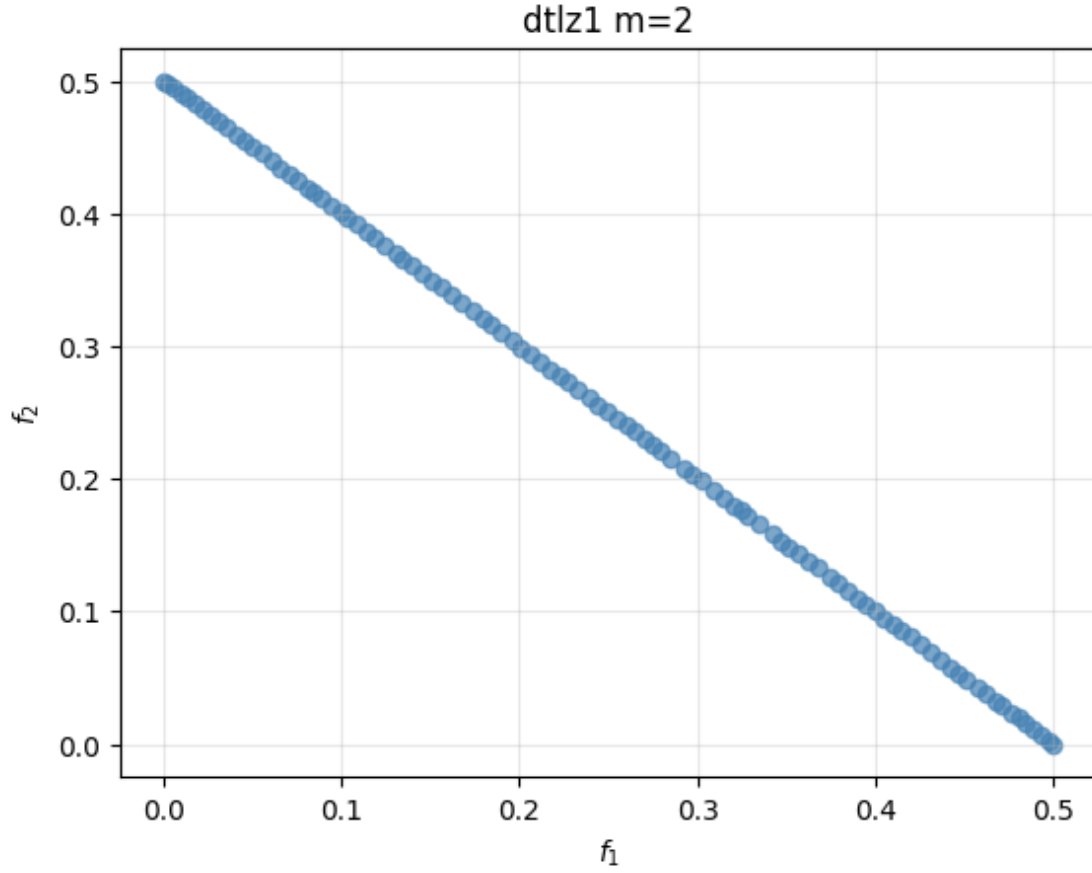


Fig. 17: Frentes Pareto (mediana HV) para DTLZ1, organizados por algoritmo y dimensionalidad. Se observa que para $m = 2$, los frentes de los tres algoritmos son similares y cercanos al frente verdadero (simplex $f_1 + f_2 = 0.5$). Para $m \geq 5$, $\mathcal{R}$ -NSGA-II produce frentes mas concentrados, indicando mejor convergencia.

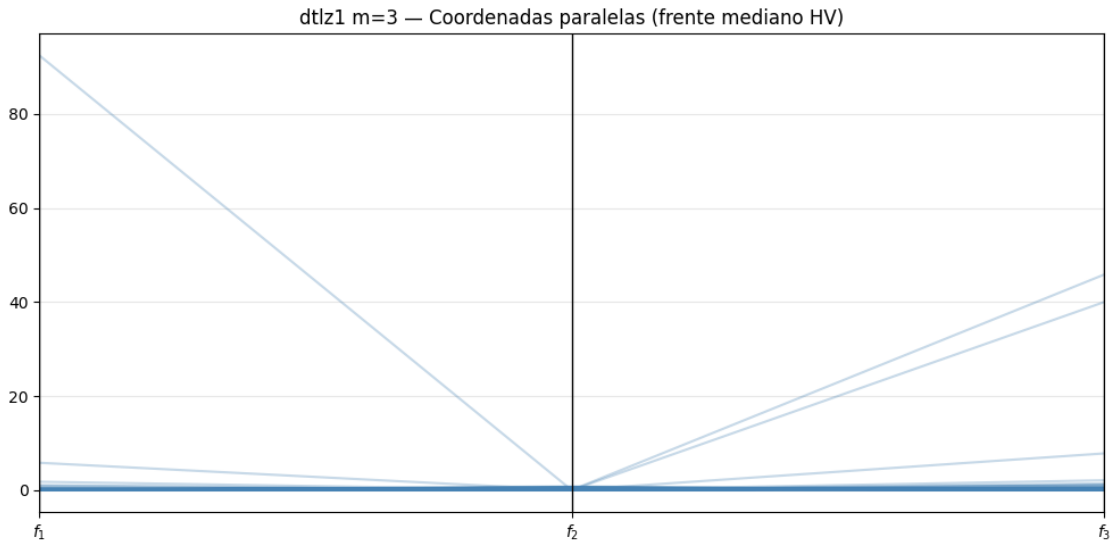


Fig. 18: Frentes Pareto (mediana HV) para DTLZ2. Los frentes para $m = 2$ muestran la curva $f_1^2 + f_2^2 = 1$ típica. Los tres algoritmos producen frentes similares en todas las dimensionalidades.

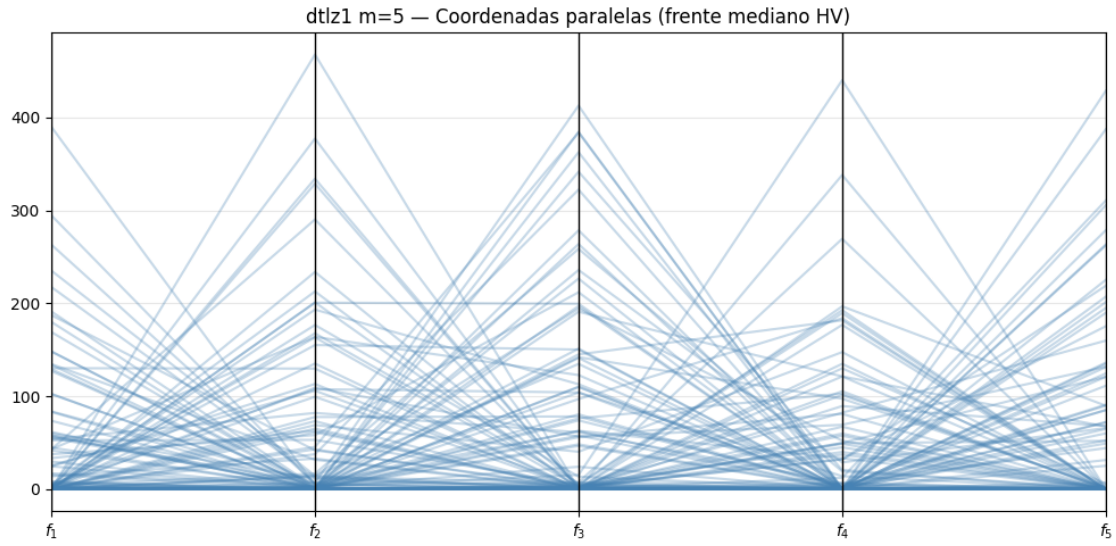


Fig. 19: Frentes Pareto (mediana HV) para DTLZ7. Se aprecian las regiones discontinuas del frente. RSE-NSGA-II muestra mejor cobertura de estas regiones en $m = 2, 3$.

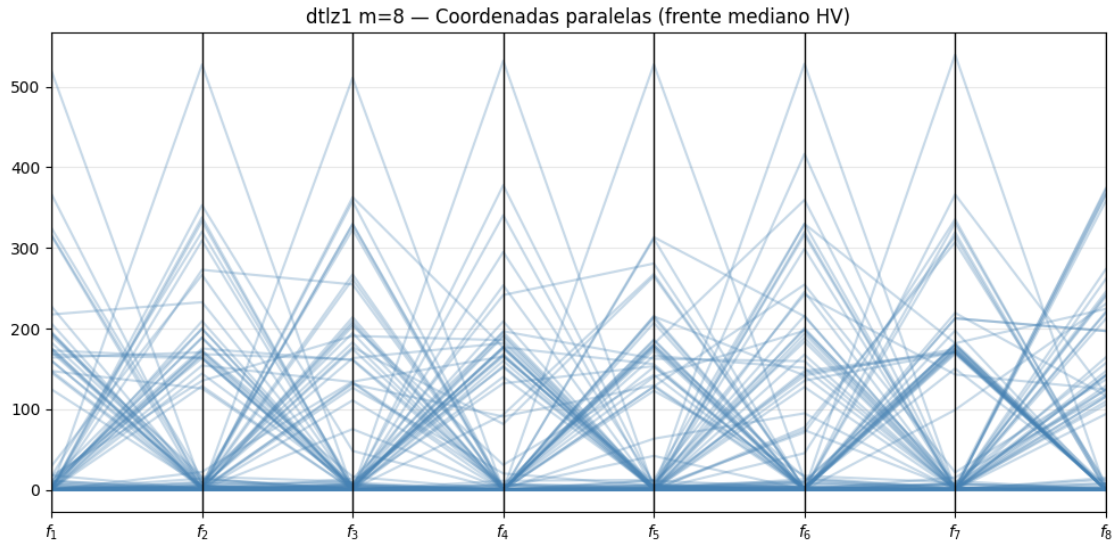


Fig. 20: Frentes Pareto (mediana HV) para WFG1. Los frentes muestran la distorsion caracteristica de WFG1 (sesgo hacia regiones del espacio). En $m = 2$, $\mathcal{R}$ -NSGA-II produce frentes mas cercanos al frente verdadero.

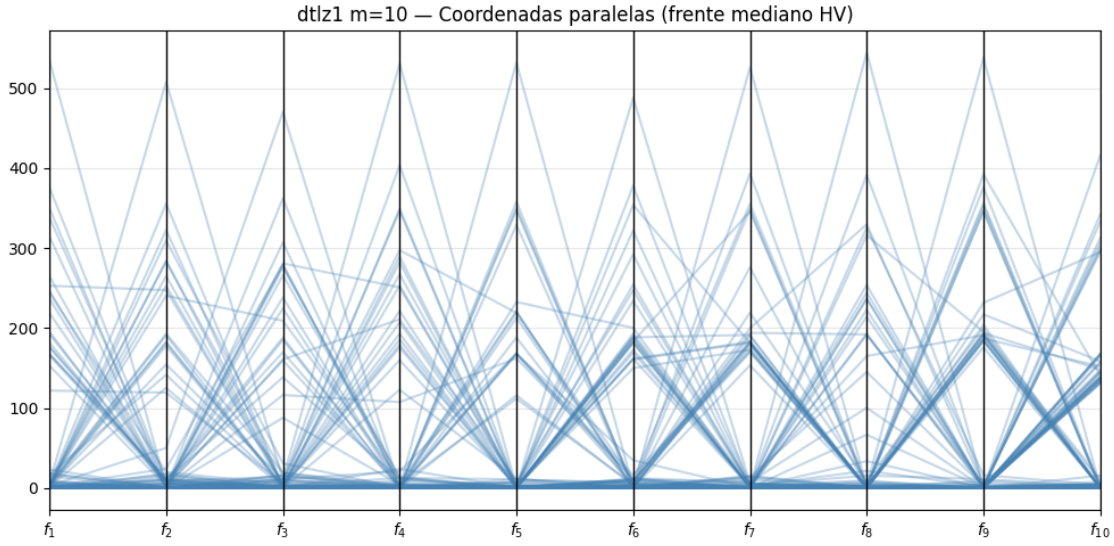


Fig. 21: Frentes Pareto (mediana HV) para WFG3. Se observa el frente lineal degenerado característico. Los tres algoritmos producen frentes similares, con RSE-NSGA-II mostrando mayor dispersión de puntos en $m = 2$.

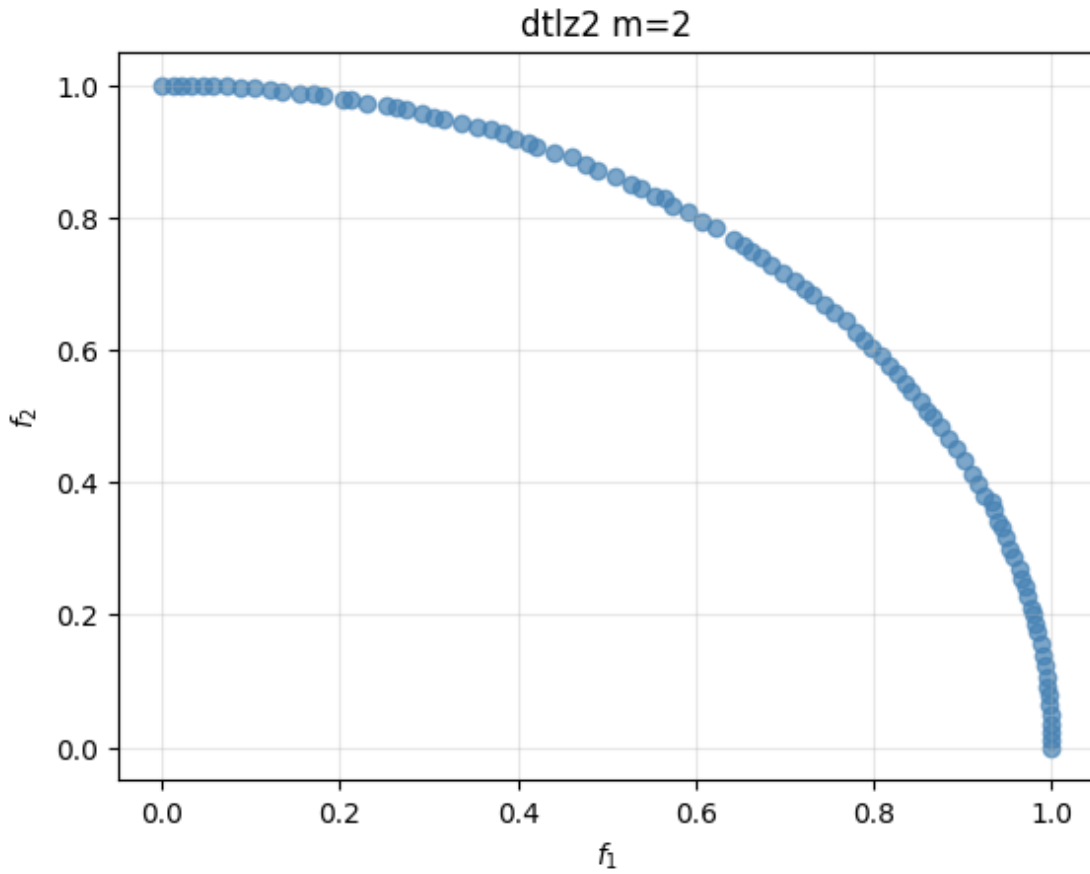


Fig. 22: Frentes Pareto (mediana HV) para IDTLZ1. Se observa que el frente converge hacia el origen ($f_i \approx 0$) para los tres algoritmos, pero $\mathcal{R}$ -NSGA-II alcanza valores más cercanos a cero, consistente con sus HV más altos.

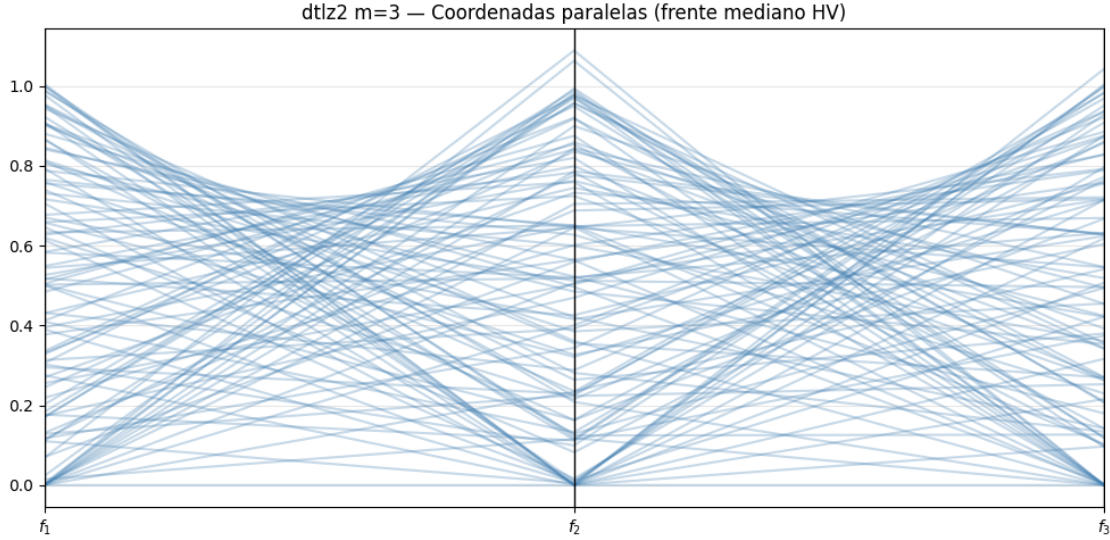


Fig. 23: Frentes Pareto (mediana HV) para IDTLZ2. Similar a IDTLZ1, los frentes convergen hacia el origen. $\mathcal{R}$ -NSGA-II produce frentes con puntos mas cercanos a cero en todas las dimensionalidades, especialmente para $m = 5, 8, 10$.

En problemas de baja dimension ($m = 2, 3$), las diferencias entre los tres algoritmos son pequenas. La dominancia de Pareto mantiene presion selectiva suficiente, y tanto crowding distance como RSE producen diversidad adecuada. RSE-NSGA-II ofrece una ventaja marginal en problemas con frentes discontinuos (DTLZ7), donde la distribucion uniforme promovida por RSE ayuda a cubrir regiones separadas.

En problemas de alta dimension ($m \geq 5$), la situacion cambia. $\mathcal{R}$ -NSGA-II con $k = 0.25$ muestra ventajas significativas en DTLZ1, IDTLZ1 e IDTLZ2, donde la (1-K)-dominancia restaura la presion selectiva que la dominancia de Pareto pierde en alta dimension. El mecanismo es claro: en $m = 10$, Pareto requiere que una solucion sea mejor o igual en las 10 objetivos para dominar a otra, lo que ocurre con probabilidad extremadamente baja. Al requerir solo $m - \lfloor 0.25m \rfloor = 8$ objetivos (para $m = 10$), la probabilidad de dominancia aumenta, generando rankings mas informativos.

Para WFG1 y WFG3, las diferencias son menores porque estos problemas tienen frentes sesgados o degenerados que son intrinsecamente difciles para cualquier algoritmo basado en dominancia. RSE-NSGA-II no muestra ventajas significativas sobre NSGA-II en la mayoria de los problemas, lo que sugiere que la mejora en diversidad no siempre se traduce en HV mas altos cuando la convergencia es el factor limitante.

El costo computacional de RSE-NSGA-II es mayor debido al calculo de la matriz de Riesz s-energy ($O(n^2)$ por generacion), que para $n = 100$ es aceptable pero se vuelve significativo para poblaciones mayores. $\mathcal{R}$ -NSGA-II, por otro lado, tiene costo similar a NSGA-II clasico pues la funcion de dominancia relajada tiene la misma complejidad que la dominancia de Pareto.

8 Plataforma y referencias

8.1 Plataforma de software y hardware

Las pruebas se realizaron en el entorno detallado en la Tabla 2. Los algoritmos ND se compilaron con Numba (@njit) para maxima eficiencia. Los algoritmos MOEA se implementaron con PyMOO 0.6.1.6, y el benchmark de 3150 ejecuciones se paralelizo con ProcessPoolExecutor usando 11 workers (12 nucleos disponibles, 1 reservado para el sistema operativo).

Table 2: Plataforma de ejecucion.

Componente	Detalle
Sistema operativo	Linux-6.14.0-33-generic-x86_64
Python	3.12.2 (packaged by conda-forge)
NumPy	2.3.5
Pandas	2.2.3
Matplotlib	3.10.7
SciPy	1.15.3
PyMOO	0.6.1.6
Nucleos de CPU	12
Algoritmos ND	Compilacion JIT con Numba (@njit)
Algoritmos MOEA	Implementacion con PyMOO
Benchmark MOEA	3150 ejecuciones en paralelo (11 workers)
Tiempo total MOEA	523.8 minutos

8.2 Referencias

References

1. J. L. Bentley, K. Clarkson, and D. B. Levine. Fast linear expected-time algorithms for computing maxima and convex hulls. *Algorithmica*, 9:168–183, 1993.
2. J. L. Bentley, H. T. Kung, M. Schkolnick, and C. D. Thompson. On the Average Number of Maxima in a Set of Vectors. *Journal of the Association of Computing Machinery*, 25(4):536–543, 1978.
3. J. L. Bentley. Multidimensional divide-and-conquer. *Commun. ACM*, 23(4):214–229, April 1980.
4. N. Drechsler, R. Drechsler, and B. Becker. Multi-objective Optimization in Evolutionary Algorithms Using Satisfiability Classes. In *Proceedings of the 6th International Conference on Computational Intelligence*, pages 108–117, 1999.
5. J. G. Falcon-Cardona, E. Covantes-Osuna, C. A. Coello Coello, and H. Ishibuchi. On the utilization of pair-potential energy functions in multi-objective optimization. *Swarm and Evolutionary Computation*, 79:101308, 2023.
6. M. Farina and P. Amato. A fuzzy definition of “optimality” for many-criteria optimization problems. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 34(3):315–326, 2004.
7. K. Ikeda, H. Kita, and S. Kobayashi. Failure of Pareto-based MOEAs: does non-dominated really mean near to optimal? In *Proceedings of the 2001 Congress on Evolutionary Computation*, volume 2, pages 957–962, 2001.
8. S. Kukkonen and J. Lampinen. Ranking-Dominance and Many-Objective Optimization. In *2007 IEEE Congress on Evolutionary Computation*, pages 3983–3990, 2007.
9. H. Sato, H. E. Aguirre, and K. Tanaka. Controlling dominance area of solutions and its impact on the performance of moeas. In *Evolutionary Multi-Criterion Optimization*, pages 5–20, 2007.
10. Y. Tian, R. Cheng, X. Zhang, Y. Su, and Y. Jin. A Strengthened Dominance Relation Considering Convergence and Diversity for Evolutionary Many-Objective Optimization. *IEEE Transactions on Evolutionary Computation*, 23(2):331–345, 2019.
11. X. Zou, Y. Chen, M. Liu, and L. Kang. A New Evolutionary Algorithm for Solving Many-Objective Optimization Problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 38(5):1402–1412, 2008.
12. J. Blank and K. Deb. Pymoo: Multi-Objective Optimization in Python. *IEEE Access*, 8:89497–89509, 2020.